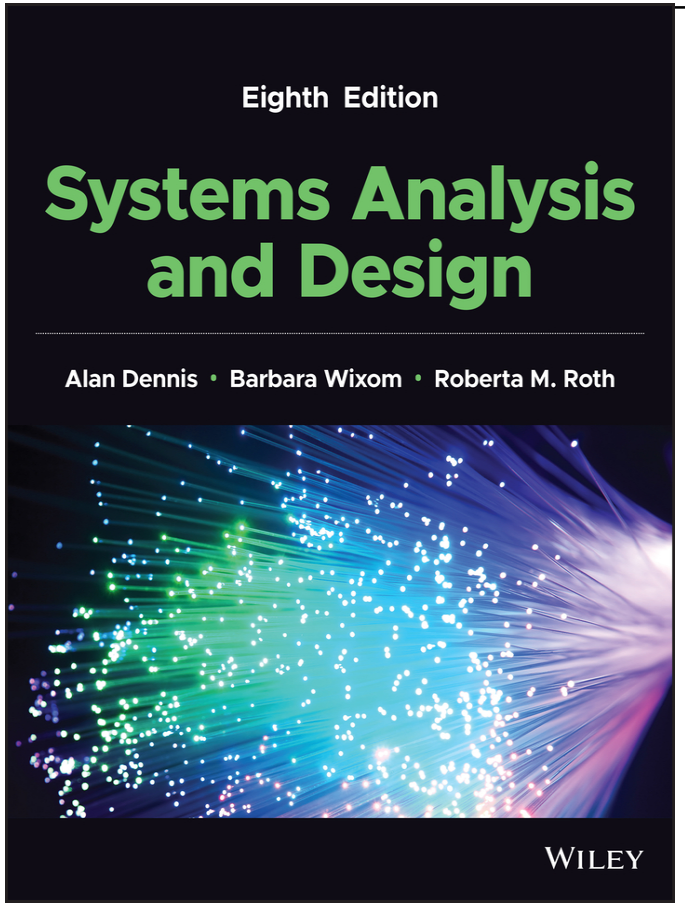
Software Development Lifecycles

Part 4



Systems Analysis and Design

Chapter 11

Moving into Implementation

Outline

- Managing the programming process.
- Testing.
- Developing documentation.

Introduction

- The **implementation phase** consists of developing and testing the system's software, documentation, and new operating procedures.
- **Managing the programming process** is the major task of the systems analyst in this phase.
- While programmers work on programming, the systems analyst design a variety of **tests** to ensure that the system does what it was designed to do.
- During this phase, the systems analysts finalize the **system documentation** and develop the **user documentation**.

Managing the Programming Process

- The project manager's tasks during the process of programming:
  - Assigning programming tasks
  - Coordinating the activities
  - Managing the programming schedule.

Assigning Programming Tasks

- The project manager first groups together modules that are related, and then assigns the groups of modules to programmers on the basis of their experience and skill level.
- The project manager must deal with a mismatch between the available programming skills and the programming skills that are needed for the project.
- More is not always better than less!
- The best size of programming team is the smallest feasible one that can function as independent as possible.
- Projects requiring a large team should be broken into a series of independent, smaller parts.

Coordinating Activities

- Coordination can be done through both high-tech and low-tech means.
- The simplest approach is to have a weekly project meeting.
- Another important method is to create and follow standards.
- Many project teams set up three “areas” for programmers:
  - development area,
  - testing area, and
  - production area.

(cont'd)

- Implement ***change control*** techniques:

  - keeping files and programs in different places according to completion status,

  - using a ***program log*** to keep track of program changes.

- Many CASE tools are set up to track the status of programs and help manage programmers.

Managing the Schedule

- The initial time estimates must be refined as the project progresses during construction.
- One of the most common causes for schedule problems is ***scope creep***.
- Another common cause is unnoticed day-by-day slippages in the schedule.
- Typically, a project manager will create risk assessment that tracks potential risks that have an impact on the schedule and costs.

(cont'd)

- Classic Implementation Mistakes:

1. Research-oriented development.
2. Using "low-cost" personnel.
3. Lack of code control.
4. Inadequate testing.

Testing

- The attention paid to testing is justified by the high costs associated with downtime and failures caused by software bugs.

Brokerage Service	$6.4 million
Energy	2.8 million
Telecom	2.0 million
Manufacturing	1.6 million
Retail	1.1 million
Health Care	636,000
Media	90,000
"Assessing the Financial Impact of Downtime," Vision Solutions, 2008, www.visionsolutions.com	

Test Planning

- Testing starts with the tester's developing a ***test plan*** that defines a series of tests that will be conducted.
- A test plan describes a set of very specific ***test cases*** to examine, and defines the expected results.
- The tester develops a series of test cases to ensure that the quality of programs is validated.
- There are four general stages of tests: unit tests, integration tests, system tests, and acceptance tests.

(cont'd)

Test plan

Test Plan Page ___ of ___

Program ID: _____ **Version number:** _____

Tester: _____ **Date designed:** _____ **Date conducted:** _____

Results: ☐ **Passed** ☐ **Open items:**

Test ID: _____ **Requirement addressed:** _____

Objective:

Test cases

Interface ID	Data Field	Value Entered
1. _____	_____	_____
2. _____	_____	_____
3. _____	_____	_____
4. _____	_____	_____
5. _____	_____	_____
6. _____	_____	_____

Script

Expected results/notes

Actual results/notes

FIGURE 11-1 Test plan.

(cont'd)

Each of the functions is a separate module that needs to be tested

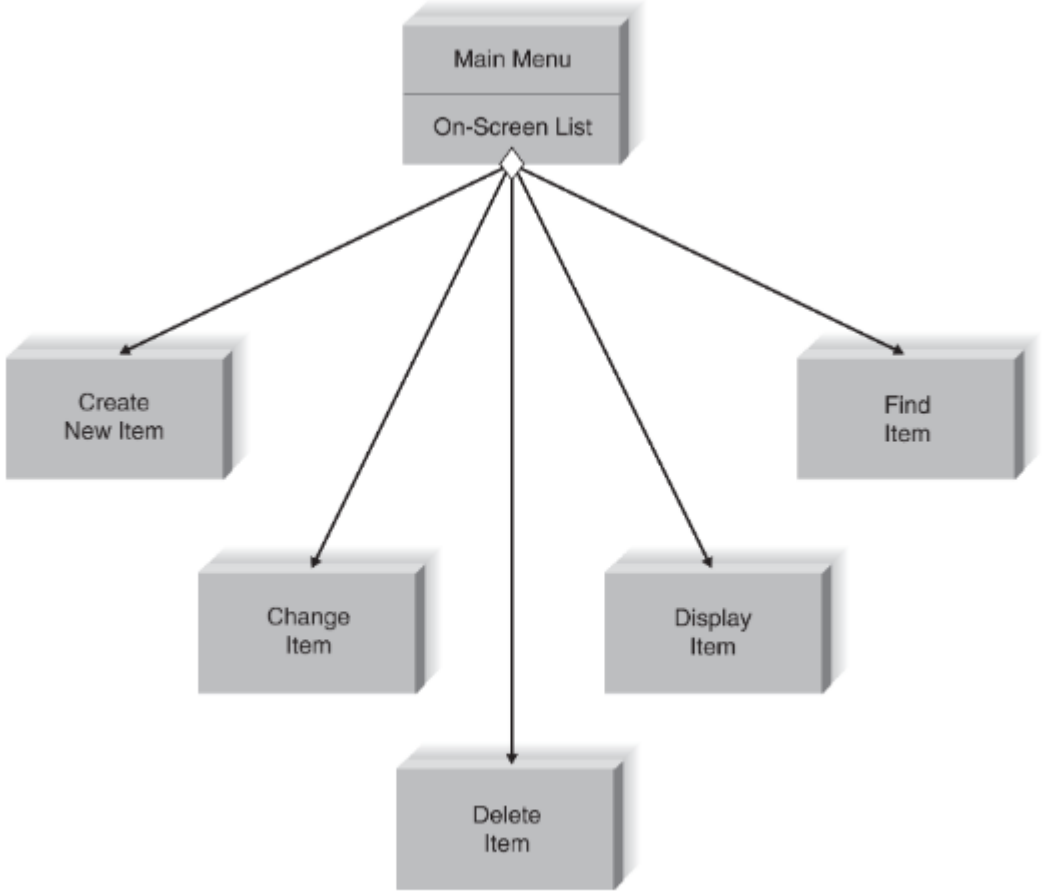


FIGURE 11-2 Testing separate modules.

(cont'd)

Error Discovery Rates for Different Stages of Tests

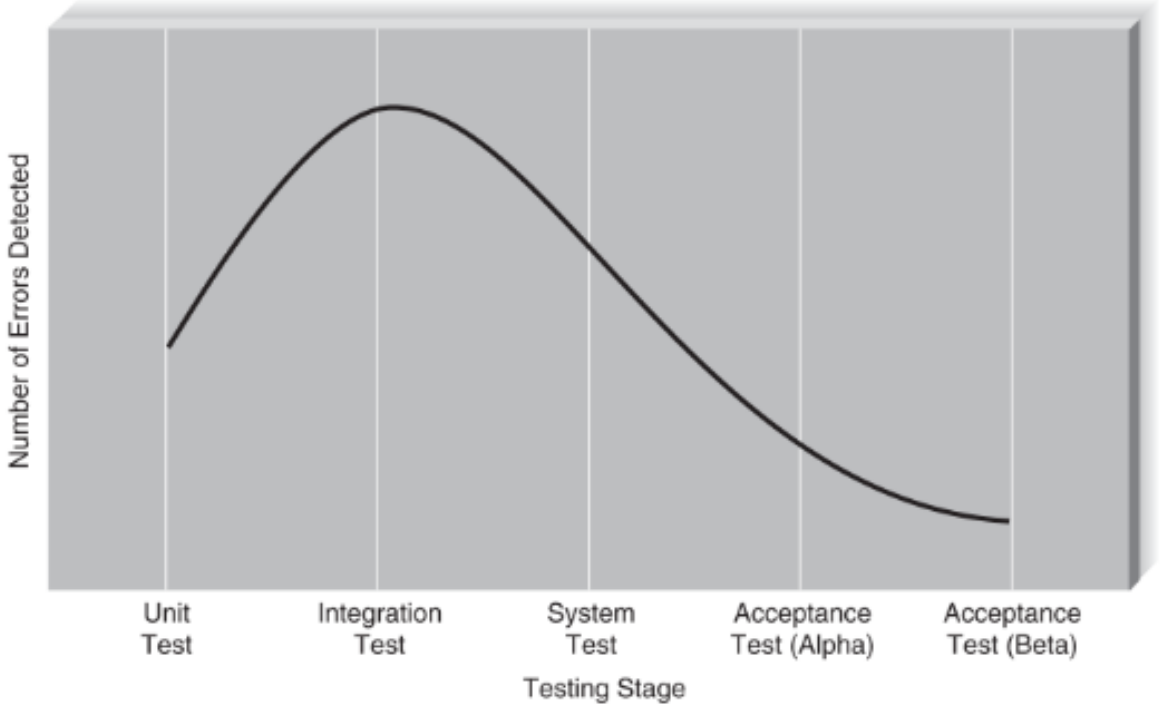


FIGURE 11-3 Error discovery rates for different stages of tests.

Unit Tests

- ***Unit tests*** focus on one unit – a program or a program module that performs a specific function that can be tested.
- There are two approaches to unit testing:

  - ***Black-box testing***
    - The test plan is developed directly from the program specification.
  - ***White-box testing***
    - The tester reviews the actual program code.

Integration Tests

- ***Integration tests*** assess whether a set of modules or programs that must work together do so without error.

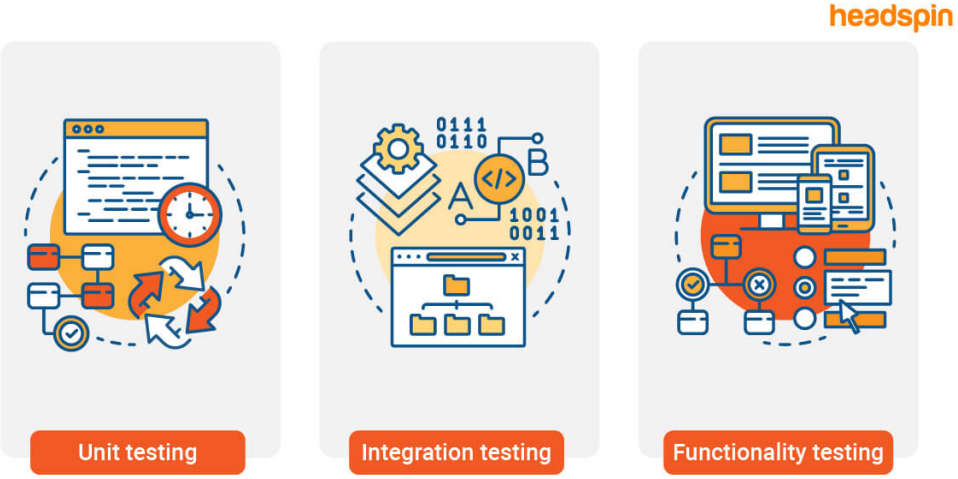
- There are four approaches to integration testing:

  - user interface testing,

  - use scenario testing,

  - data flow testing, and

  - system interface testing.



System Tests

- ***System tests*** are usually conducted by the systems analysts to ensure that all modules and programs work together without error.
- System tests examine

  - how well the system meets business requirements,

  - usability,

  - security,

  - performance under heavy load, and

  - system's documentation.

Acceptance Tests

- ***Acceptance tests*** are done primarily by the users.
- The goal of acceptance tests is to confirm that the system is complete, meets the business needs, and is acceptable to the users.
- Acceptance testing is done in two stages:

- alpha testing - users test the system using made-up data,

- beta testing – users begin to use the system with real data and carefully monitor the system for errors.

(cont'd)

Types of Tests

Stage	Types of Tests	Test Plan Source	When to Use	Notes
Unit Testing	**Black-box testing:** treats program as black box.	Program specifications	For normal unit testing	The tester focuses on whether the unit meets the requirements stated in the program specifications.
	White-box testing: looks inside the program to test its major elements.	Program source code	When complexity is high	By looking inside the unit to review the code itself, the tester may discover errors or assumptions not immediately obvious to someone treating the unit as a black box.
Integration Testing	**User interface testing:** The tester tests each interface function.	Interface design	For normal integration testing	Testing is done by moving through each and every menu item in the interface either in a top-down or bottom-up manner.
	Use scenario testing: The tester tests each use scenario.	Use scenario	When the user interface is important	Testing is done by moving through each use scenario to ensure that it works correctly. Use scenario testing is usually combined with user interface testing because it does not test all interfaces.
	Data flow testing: Tests each process in a step-by-step fashion.	Physical DFDs	When the system performs data processing	The entire system begins as a set of stubs. Each unit is added in turn, and the results of the unit are compared with the correct result from the test data; when a unit passes, the next unit is added and the test is rerun.
	System interface testing: tests the exchange of data with other systems.	Physical DFDs	When the system exchanges data	Because data transfers between systems are often automated and not monitored directly by the users, it is critical to design tests to ensure that they are being done correctly.
System Testing	**Requirements testing**: tests whether original business requirements are met.	System design, unit tests, and integration tests	For normal system testing	This test ensures that changes made because of integration testing did not create new errors. Testers often pretend to be uninformed users and perform improper actions to ensure that the system is immune to invalid actions (e.g., adding blank records).
	Usability testing: tests how convenient the system is to use.	Interface design and use scenarios	When user interface is important	This test is often done by analysts with experience in how users think and in good interface design. This test sometimes uses the formal usability testing procedures discussed in Chapter 8.
	Security testing: tests disaster recovery and unauthorized access.	Infrastructure design	When the system is important	Security testing is a complex task, usually done by an infrastructure analyst assigned to the project. In extreme cases, a professional firm may be hired.
	Performance testing: examines the ability to perform under high loads.	System proposal and infrastructure design	When the system is important	High volumes of transactions are generated and given to the system. This test is often done using special-purpose testing software.
	Documentation testing: tests the accuracy of the documentation.	Help system, procedures, tutorials	For normal system testing	Analysts spot-check or check every item on every page in all documentation to ensure that the documentation items and examples work properly.
Acceptance Testing	**Alpha testing:** conducted by users to ensure that they accept the system.	System tests	For normal acceptance testing	Alpha tests often repeat previous tests but are conducted by users themselves to ensure that they accept the system.
	Beta testing: uses real data, not test data.	No plan	When the system is important	Users closely monitor the system for errors or useful improvements

DFD = data flow diagram.

FIGURE 11-4 Types of tests.

Developing Documentation

- There are two fundamentally different types of documentation:

- ***system documentation*** is intended to help programmers and systems analysts understand the system and enable them to build it or maintain it;

- ***user documentation*** is designed to help the user operate the system.

(cont'd)

- User documentation should not be left until the end of the project.
- Time required to develop and test user documentation should be built into project plan.
- On-line documentation is becoming the predominant form.

(cont'd)

- There are four key strengths of online documentation:

- Searching for information is simpler.

- The same information can be presented in many different formats.

- It enables the user to interact with the documentation in many new ways.

- It is significantly less expensive than paper documentation.

Types of User Documentation

- There are three fundamentally different types of user documentation:

- ***Reference Documents*** are designed to be used when the user needs to learn how to perform a specific function.

- ***Procedural Manuals*** describe how to perform business task.

- ***Tutorials*** teach people how to use major components of the system.

Designing Documentation Structure

- The general structure used in online documentation is to develop a set of ***documentation navigation controls*** that lead the user to ***documentation topics***.
- There are five general types of navigation controls for topics: table of contents, index, text search, intelligent agent, and Web-like links.

(cont'd)

Organizing
Online
Reference
Documents

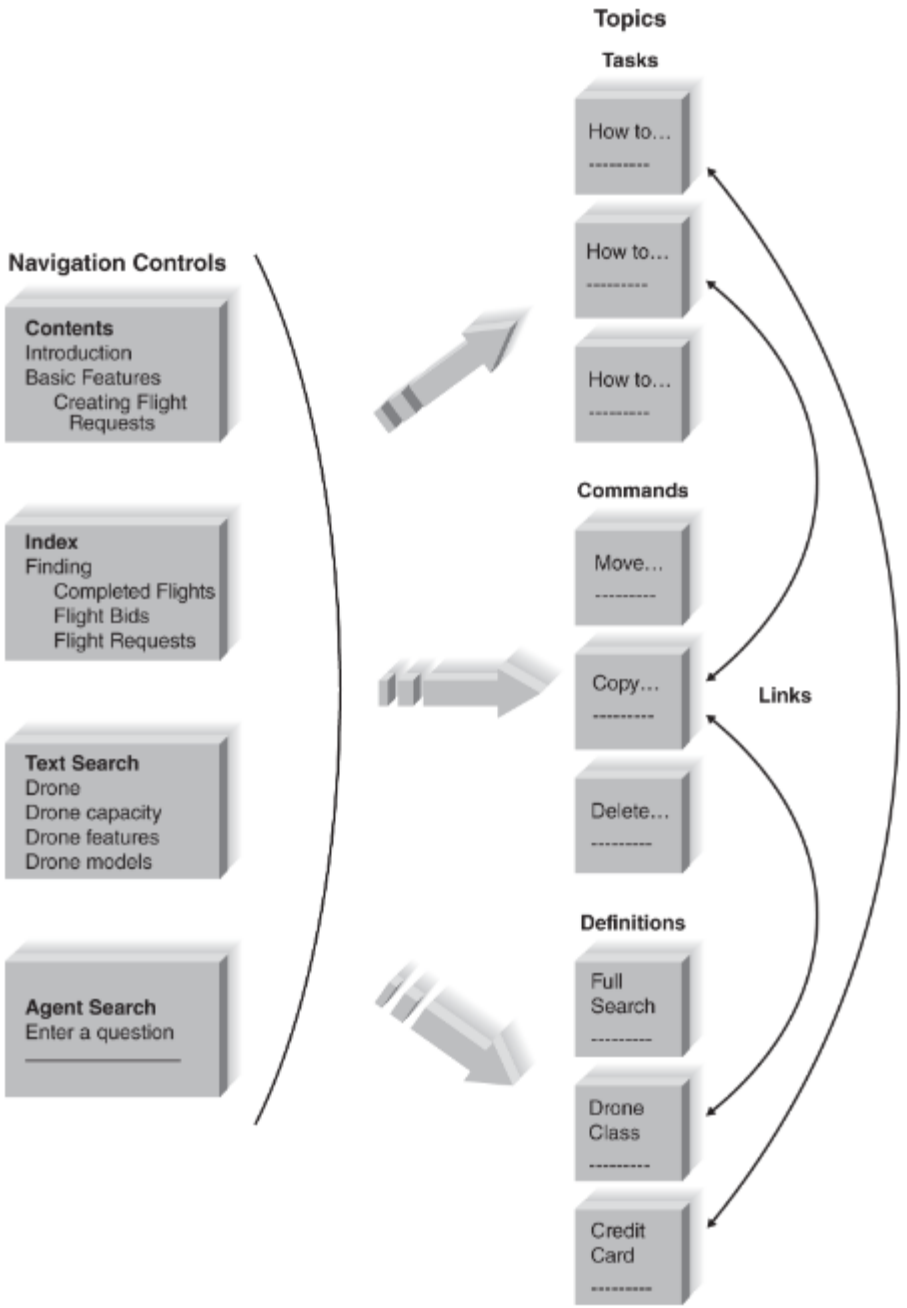


FIGURE 11-5 Organizing online reference documents.

Writing Documentation Topics

- Topics start with clear titles, followed by introductory text that define the topic, and then provide detailed, step-by-step instructions.
- Many topics include screen images and “show me” examples.
- Most also include navigation controls to enable movement among topics.
- Some also have links to related topics.

(cont'd)

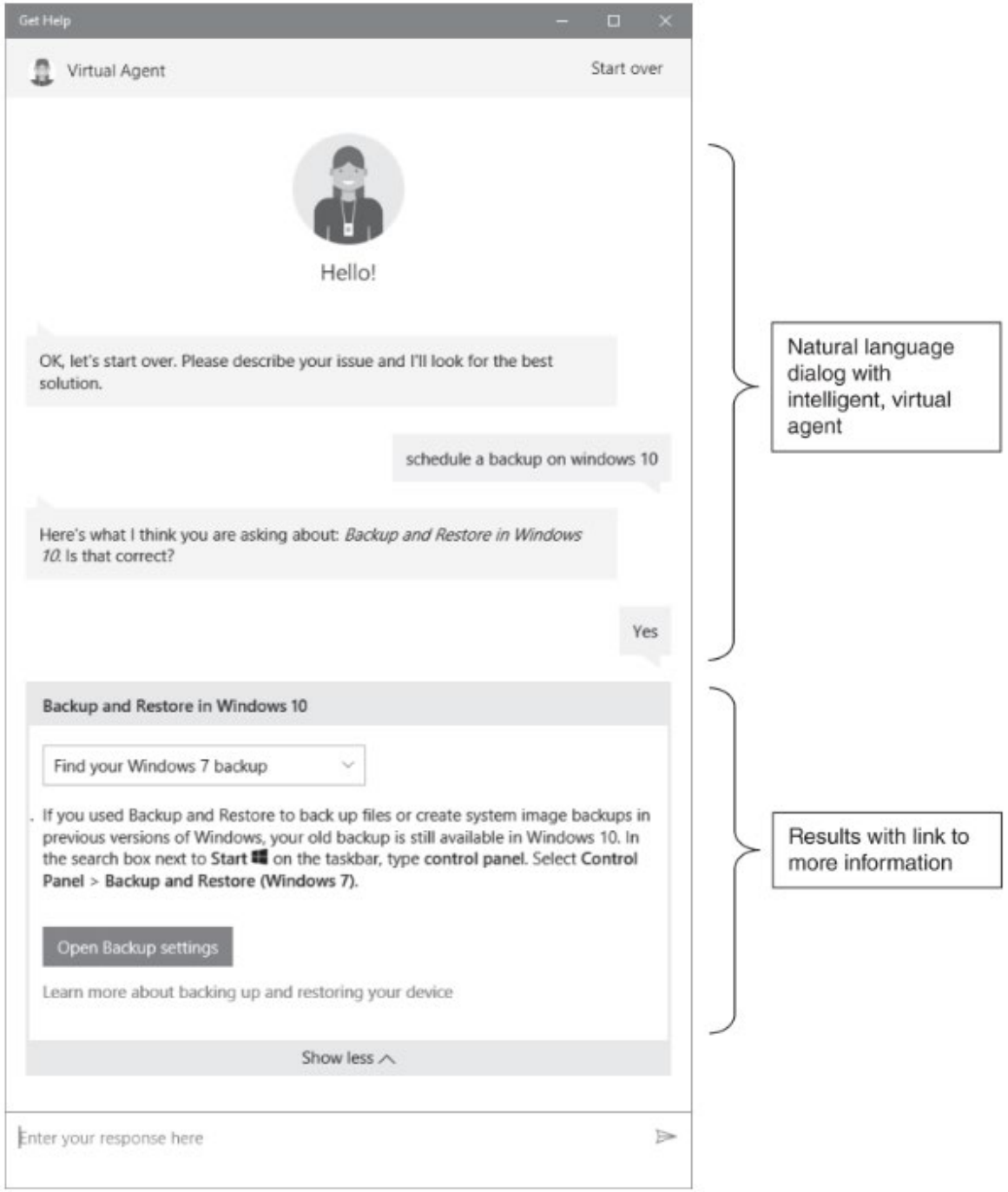


FIGURE 11-6 Virtual agent help dialog in Windows 10.

Identifying Navigation Terms

- ***Navigation terms*** are used to help users find topics.

- Terms for the index and search engine can come from four distinct sources:

- The set of the commands in the user interface;

- The set of major concepts in the system (e.g., data entities);

- The set of business tasks the user performs;

- The set of synonyms for the three sets of items mentioned above.

Applying the Concepts at DrōnTeq

- Managing Programming
- Testing
- Developing User Documentation

Test Stage	Client-Focused Site	Pilot-Focused Site
Unit tests	Black-box tests	Black-box and white-box tests
Integration tests	User interface tests; use scenario tests	User interface tests; use scenario tests
System tests	Requirements tests; security tests; performance tests; usability tests	Requirements tests; security tests; performance tests; usability tests
Acceptance tests	Alpha test; beta test	Alpha test; beta test

FIGURE 11-8 Flight services test plan.

SUMMARY

- **Managing programming**

- The project manager assigns tasks to the programmers, coordinates the program development, manages the schedule, and adjusts the schedule.

- **Testing**

- A test plan contains several tests.

- A test specifies several test cases.

- Four types of tests: unit tests, integration tests, system tests, and acceptance tests.

- **Documentation**

- User documentation and system documentation are moving towards online documentation.

- Three types of user documentation: reference documents, manuals, and tutorials.

BTEC Alliance with FPT Education Pearson